

FREE LESSON · FROM MODULE 04



Build a Studio-Grade Intro Screen in 10 Minutes

YOU'LL SHIP

A black-screen logo intro with a smooth fade and scale pop

Launch any big game and the first thing you see is a black screen with the studio's logo. It's a small detail, but it instantly makes a game feel finished. By the end of this lesson your game will have:

- a black screen the moment the game starts loading,
- your logo fading in at the center with a subtle scale "pop",
- a clean fade-out after 2 seconds, straight into gameplay.

You'll need: Roblox Studio and a logo image (ideally a square PNG with a black or transparent background).

Step 1: Upload your logo (2 min)

1. Open your game in Roblox Studio.
2. Go to **View** → **Asset Manager**.
3. Click **Import** and pick your logo file.
4. Right-click the uploaded image → **Copy Asset ID**.

The ID looks like `rbxassetid://1234567890`. Keep it handy.

Uploaded images go through Roblox moderation before they show up in-game. This usually takes a few minutes.

Step 2: Create the script in the right place (1 min)

1. In the **Explorer**, find **ReplicatedFirst**.
2. Right-click it → **Insert Object** → **LocalScript**.
3. Rename it to `IntroSplash`.

Why ReplicatedFirst? Everything inside it is sent to the player and run first, before the rest of the game has loaded. That makes it the only right place for an intro screen.

Why a LocalScript? The screen belongs to one specific player, not the whole server. Code that draws UI runs on the player's device (the client).

Step 3: Paste the code (2 min)

Paste this into the script, then replace `YOUR_ASSET_ID` on the `LOGO_IMAGE` line with your logo's ID.

```
local ReplicatedFirst = game:GetService("ReplicatedFirst")
local Players = game:GetService("Players")
local TweenService = game:GetService("TweenService")

-- Hide Roblox's default loading screen
ReplicatedFirst:RemoveDefaultLoadingScreen()
```

```

local LOGO_IMAGE = "rbxassetid://YOUR_ASSET_ID"

local FADE_IN_TIME = 0.25
local LOGO_ON_SCREEN_TIME = 2
local FADE_OUT_TIME = 0.4

local player = Players.LocalPlayer
local playerGui = player:WaitForChild("PlayerGui")

-- The ScreenGui is the canvas we draw on
local screenGui = Instance.new("ScreenGui")
screenGui.Name = "IntroSplash"
screenGui.IgnoreGuiInset = true -- cover Roblox's top bar too
screenGui.DisplayOrder = 1000 -- render above every other UI
screenGui.ResetOnSpawn = false -- survive the player respawning

-- Full-screen black background
local background = Instance.new("Frame")
background.Size = UDim2.new(1, 0, 1, 0)
background.BackgroundColor3 = Color3.new(0, 0, 0)
background.BorderSizePixel = 0
background.Parent = screenGui

-- Logo in the center
local logo = Instance.new("ImageLabel")
logo.Image = LOGO_IMAGE
logo.BackgroundTransparency = 1
logo.AnchorPoint = Vector2.new(0.5, 0.5)
logo.Position = UDim2.new(0.5, 0, 0.5, 0)
logo.Size = UDim2.new(0.4, 0, 0.4, 0)
logo.ImageTransparency = 1 -- start invisible
logo.Parent = background

-- Keep the logo square on every screen, desktop or mobile
local aspect = Instance.new("UIAspectRatioConstraint")
aspect.AspectRatio = 1
aspect.Parent = logo

-- UIScale drives the "pop" without touching Size
local logoScale = Instance.new("UIScale")
logoScale.Scale = 0.9
logoScale.Parent = logo

```

```

screenGui.Parent = playerGui

-- Fade in (transparency 1 -> 0) and pop (scale 0.9 -> 1)
TweenService:Create(logoScale, TweenInfo.new(FADE_IN_TIME, Enum.EasingStyle.Back), {
Scale = 1 }):Play()
local fadeIn = TweenService:Create(logo, TweenInfo.new(FADE_IN_TIME), { ImageTransparency
= 0 })
fadeIn:Play()
fadeIn.Completed:Wait()

-- Keep the logo up for 2 seconds in total
task.wait(LOGO_ON_SCREEN_TIME - FADE_IN_TIME)

-- Fade out the logo and the background together
local fadeOutInfo = TweenInfo.new(FADE_OUT_TIME)
TweenService:Create(logo, fadeOutInfo, { ImageTransparency = 1 }):Play()
local bgFadeOut = TweenService:Create(background, fadeOutInfo, { BackgroundTransparency =
1 })
bgFadeOut:Play()
bgFadeOut.Completed:Wait()

-- Clean up so nothing blocks gameplay
screenGui:Destroy()

```

Step 4: Test it (1 min)

Hit **Play** (F5). You should see a black screen, your logo animating in, and then the game.

How it works: 3 things to remember

- 1. Scale vs. Offset.** `UDim2.new(0.5, 0, 0.5, 0)` means "50% of the width, 50% of the height". In each pair, the first number is a fraction of the screen (Scale) and the second is pixels (Offset). Scale behaves the same on every screen size, which is why we use it here.
- 2. AnchorPoint.** By default, UI is positioned from its top-left corner. `AnchorPoint = (0.5, 0.5)` says "position me by my center". Combined with a 50% / 50% position, the logo sits dead center.
- 3. TweenService.** Instead of changing transparency in a loop, you tell TweenService *what* to change, *to what value*, and *over how long*, and it animates smoothly for you. You'll use tweens for most UI animation from here on.

Challenges

1. Change `LOGO_ON_SCREEN_TIME` to 3 and feel how the pacing changes.
2. Add a `TextLabel` with your game's name under the logo that fades in with it. (Hint: tween `TextTransparency`.)
3. Try a different `EasingStyle`, like `Enum.EasingStyle.Bounce`.

What's next

This lesson is from Module 4 of **PIXEL KIT: Learn Roblox Studio, Ship Real Games**. In the full course you'll take one game from an empty baseplate to launch: world building, scripting, UI, saving player data, an in-game store, and publishing.

PIXEL KIT is not affiliated with or endorsed by Roblox Corporation.